Django Request Lifecycle — Step by Step

1. What Is the Django Request Lifecycle?

The **Django Request Lifecycle** describes the complete process through which an HTTP request travels from the user's browser to the Django application, gets processed, and returns an HTTP response to the browser.

সহজভাবে বললে, একজন user যখন browser-এ কোনো URL প্রবেশ করে, তখন Django কীভাবে সেই request গ্রহণ করে, process করে এবং response পাঠায়—এই সম্পূর্ণ প্রক্রিয়াকে Django Request Lifecycle বলে।

Complete Flow

```
Browser
   |
   v
HTTP Request
   |
   v
Web Server
   |
   v
WSGI / ASGI
   |
   v
Request Middleware
   |
   v
URL Resolver (urls.py)
   |
   v
View (views.py)
   |
   +------> Model / ORM ------> Database
   |               ^
   |               |
   |         Query Result
   |
   +------> Template Engine (if needed)
   |
   v
HTTP Response
```

```
   |
   v
Response Middleware
   |
   v
WSGI / ASGI
   |
   v
Web Server
   |
   v
Browser
```

Note: Database এবং Template Engine প্রতিটি request-এর জন্য বাধ্যতামূলক নয়। API response তৈরি করলে template প্রয়োজন নাও হতে পারে, আবার কোনো request-এ database query-ও প্রয়োজন নাও হতে পারে।

---

2. User Sends an HTTP Request

প্রথমে user browser-এ একটি URL প্রবেশ করে।

Example:

```
https://example.com/products/5/
```

Browser এই URL-এর জন্য একটি HTTP request তৈরি করে এবং server-এ পাঠায়।

একটি HTTP request-এ থাকতে পারে:

- **HTTP Method:** `GET`, `POST`, `PUT`, `PATCH`, `DELETE`
- **URL Path:** `/products/5/`
- **Headers:** Request সম্পর্কে অতিরিক্ত তথ্য
- **Cookies:** Browser-এ সংরক্ষিত cookie
- **Query Parameters:** URL-এর অতিরিক্ত parameter
- **Request Body:** `POST`, `PUT` বা `PATCH` request-এর data

Example HTTP Request

```
GET /products/5/ HTTP/1.1
Host: example.com
Cookie: sessionid=abc123
```

এখানে:

- `GET` — HTTP method
- `/products/5/` — URL path

- `Host` — Server-এর hostname
- `Cookie` — Browser থেকে পাঠানো cookie

---

3. Web Server and WSGI / ASGI

Browser থেকে request প্রথমে web server-এর কাছে পৌঁছায়। Server এবং Django application-এর মধ্যে WSGI বা ASGI interface request আদান-প্রদানে সাহায্য করে।

WSGI

WSGI = Web Server Gateway Interface

WSGI হলো Python web application-এর জন্য একটি traditional synchronous interface।

Examples:

- Gunicorn
- uWSGI

ASGI

ASGI = Asynchronous Server Gateway Interface

ASGI synchronous এবং asynchronous উভয় ধরনের application handling সমর্থন করে। এটি WebSocket-এর মতো protocol-ও সমর্থন করতে পারে।

Examples:

- Daphne
- Uvicorn
- Hypercorn

WSGI	ASGI
Traditional synchronous interface	Synchronous ও asynchronous handling সমর্থন করে
Traditional HTTP request-response	Async applications এবং WebSock

applications-এর জন্য উপযোগী	et-এর মতো protocol সমর্থন করতে পারে
Async capability সীমিত	Async capability বেশি

Important: ASGI ব্যবহার করলেই Django application-এর সব code automatically asynchronous হয়ে যায় না।

---

4. Django Middleware — Request Phase

Django application-এ request প্রবেশ করার পর middleware chain request process করে।

Middleware হলো এমন একটি reusable layer, যা view execute হওয়ার আগে request process করতে পারে এবং view থেকে response ফেরার পর response process করতে পারে।

Common Django middleware:

- `SecurityMiddleware`
- `SessionMiddleware`
- `CommonMiddleware`
- `CsrfViewMiddleware`
- `AuthenticationMiddleware`
- `MessageMiddleware`

Request Flow

```
Request
   |
   v
Middleware 1
   |
   v
Middleware 2
   |
   v
Middleware 3
   |
   v
URL Resolver
```

Middleware যেসব কাজ করতে পারে:

1. Request information process করা।
2. Security checks সম্পন্ন করা।
3. Session information load করা।
4. Authentication-related information যোগ করা।
5. CSRF protection পরিচালনা করা।
6. প্রয়োজন হলে request-এর processing বন্ধ করে response ফেরত দেওয়া।

Example: কোনো middleware request reject করলে request আর view পর্যন্ত নাও পৌঁছাতে পারে।

---

5. URL Resolver — `urls.py`

Middleware processing-এর পর Django requested URL-এর সঙ্গে URL patterns match করে। এই কাজটি URL resolver করে।

Example: `urls.py`

```
from django.urls import path
from .views import product_detail

urlpatterns = [
    path("products/<int:id>/", product_detail, name="product_detail"),
]
```

ধরা যাক, browser request করেছে:

```
/products/5/
```

Django URL pattern-এর সঙ্গে URL match করবে:

```
path("products/<int:id>/", product_detail, name="product_detail")
```

এখানে:

```
products/       → Static URL segment
<int:id>        → Integer URL parameter
5               → id-এর value
product_detail  → যে view execute হবে
```

ফলে Django view-কে `id=5` argument দিয়ে call করবে।

যদি URL Match না করে?

কোনো URL pattern match না করলে সাধারণত Django `404 Not Found` response তৈরি করে।

---

6. View Execution — `views.py`

URL resolver নির্ধারণ করার পর Django সংশ্লিষ্ট view execute করে।

View হলো Django application-এর এমন একটি component, যা request গ্রহণ করে, application logic execute করে এবং একটি HTTP response return করে।

Example: `views.py`

```
from django.shortcuts import render
from .models import Product


def product_detail(request, id):
    product = Product.objects.get(id=id)

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

এই view-এর কাজ:

1. `request` গ্রহণ করা।
2. URL থেকে পাওয়া `id` গ্রহণ করা।
3. Database থেকে সংশ্লিষ্ট product খুঁজে বের করা।
4. Product-এর data template-এ পাঠানো।
5. Render করা HTTP response return করা।

Django-তে প্রধানত দুই ধরনের view দেখা যায়।

Function-Based View (FBV)

```
from django.http import HttpResponse


def hello(request):
    return HttpResponse("Hello, World!")
```

Class-Based View (CBV)

```
from django.views.generic import DetailView
from .models import Product
```

```
class ProductDetailView(DetailView):
    model = Product
    template_name = "product.html"
```

FBV function দিয়ে এবং CBV class দিয়ে view logic তৈরি করে।

---

7. Model, ORM and Database

View-এর application logic অনুযায়ী database থেকে data প্রয়োজন হলে Django ORM ব্যবহার করতে পারে।

ORM = Object-Relational Mapper

Django ORM ব্যবহার করে Python code দিয়ে database-এর data query, create, update এবং delete করা যায়।

Example

```
product = Product.objects.get(id=5)
```

এই code-এর মাধ্যমে Django `id=5` -এর product খুঁজে বের করে।

সাধারণ flow:

```
View
  |
  v
Django ORM
  |
  v
SQL Query
  |
  v
Database
  |
  v
Query Result
  |
  v
Python Object
  |
  v
View
```

ধরা যাক, database-এ product-এর data রয়েছে:

id	name	price
5	Laptop	75000

তাহলে:

```
product = Product.objects.get(id=5)

print(product.name)
print(product.price)
```

Output:

```
Laptop
75000
```

Important: `Product.objects.get(id=5)` কোনো matching object না পেলে `Product.DoesNotExist` exception raise করে। একাধিক object match হলে `Product.MultipleObjectsReturned` exception হয়।

---

8. Template Rendering

যখন Django-কে HTML page তৈরি করতে হয়, তখন template engine ব্যবহার করা যেতে পারে।

View থেকে `render()` function-এর মাধ্যমে template এবং context data পাঠানো হয়।

View

```
from django.shortcuts import render
from .models import Product


def product_detail(request, id):
    product = Product.objects.get(id=id)

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

Template: `product.html`

```
<h1>{{ product.name }}</h1>
<p>Price: {{ product.price }}</p>
```

Context Data

```
{"product": product}
```

Rendering Flow

```
Template + Context Data
          |
          v
    Django Template Engine
          |
          v
      Rendered HTML
```

Django template engine `{{ product.name }}` এবং `{{ product.price }}`-এর মতো variable-গুলোকে সংশ্লিষ্ট data দিয়ে render করে।

Example output:

```
<h1>Laptop</h1>
<p>Price: 75000</p>
```

Important: `render()` template render করে একটি response object return করে। এটি নিজে সরাসরি browser-এ HTML পাঠায় না; Django-এর response-handling process-এর মাধ্যমে response browser-এ পৌঁছায়।

---

9. HTTP Response Is Created

View শেষ পর্যন্ত একটি HTTP response return করে।

Django-তে বিভিন্ন ধরনের response class রয়েছে।

Response	**ব্যবহার**
`HttpResponse`	সাধারণ text বা HTML response

`JsonResponse`	JSON response, বিশেষ করে API-তে
`HttpResponseRedirect`	Redirect response
`StreamingHttpResponse`	Streaming response
`TemplateResponse`	Template rendering-ভিত্তিক response

Example 1: `HttpResponse`

```
from django.http import HttpResponse


def hello(request):
    return HttpResponse("Hello, World!")
```

Example 2: `JsonResponse`

```
from django.http import JsonResponse


def api(request):
    return JsonResponse({
        "name": "Faruk",
        "age": 25,
    })
```

Output:

```
{
    "name": "Faruk",
```

```
    "age": 25
}
```

Example 3: Redirect

```
from django.shortcuts import redirect


def my_view(request):
    return redirect("home")
```

এখানে `"home"` হলো URL pattern-এর name। Django সাধারণত redirect response-এ `Location` header পাঠায়, যার মাধ্যমে browser অন্য URL-এ request করতে পারে।

---

10. Middleware — Response Phase

View থেকে response return হওয়ার পর response middleware chain-এর মধ্য দিয়ে ফিরে আসে।

```
                Request
                   |
                   v
            Middleware 1
                   |
                   v
            Middleware 2
                   |
                   v
            Middleware 3
                   |
                   v
                View
                   |
                   v
            Middleware 3
                   |
                   v
            Middleware 2
                   |
                   v
            Middleware 1
                   |
                   v
                Response
```

Middleware response-এর ওপর যেসব কাজ করতে পারে:

- Response headers যোগ করা।
- Cookies set করা।
- Security-related processing করা।
- Response content বা status পরিবর্তন করা।
- Caching-related processing করা।

Middleware-এর response processing সাধারণত request processing-এর বিপরীত ক্রমে চলে।

---

11. Web Server Sends the Response to the Browser

Django-এর response processing শেষ হলে response server/interface layer-এর মাধ্যমে browser-এ পাঠানো হয়।

```
Django Application
        |
        v
   WSGI / ASGI
        |
        v
    Web Server
        |
        v
      Browser
```

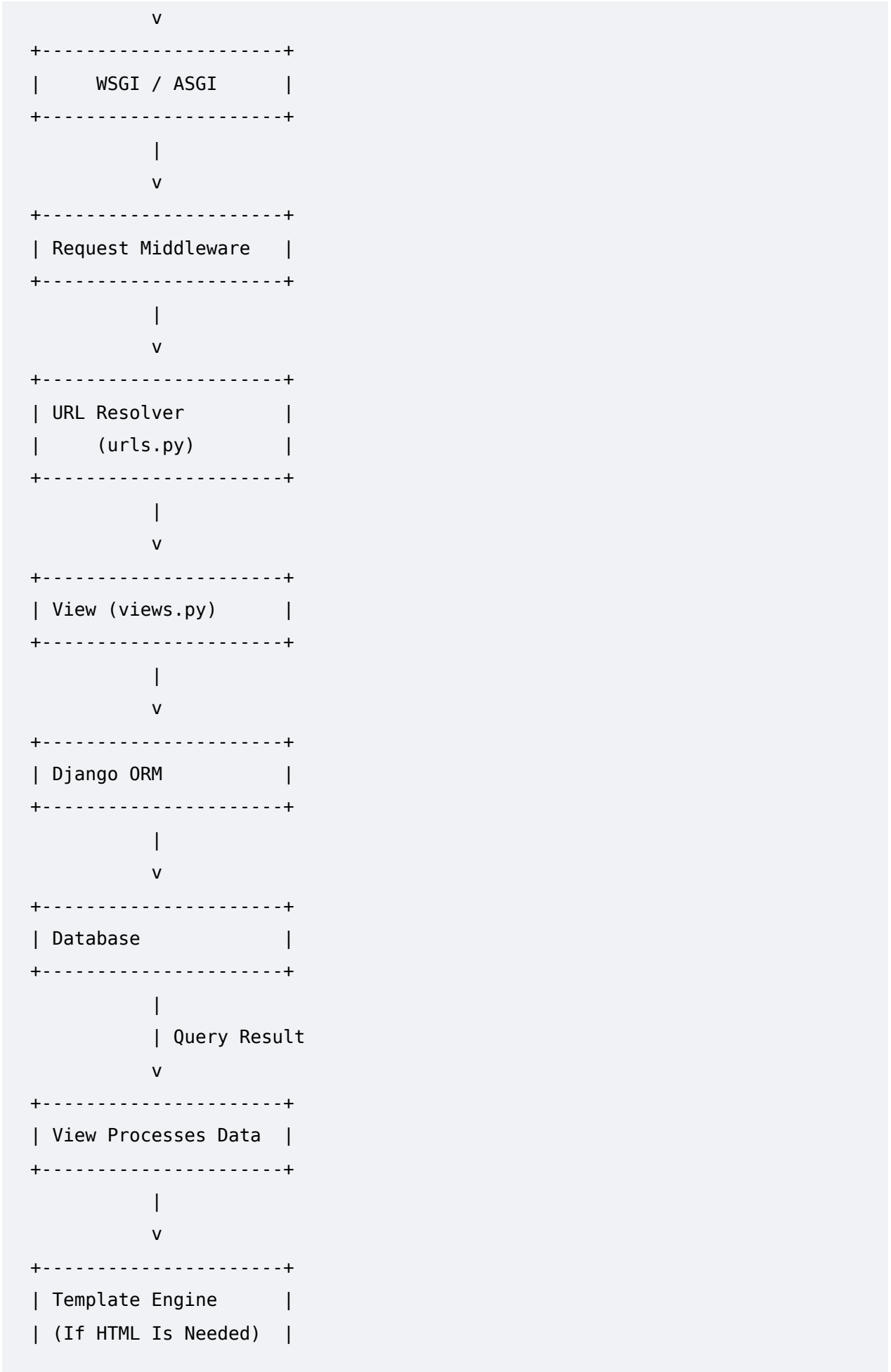
Browser response গ্রহণ করে। Response যদি HTML হয়, তাহলে browser HTML render করে user-কে page দেখায়।

JSON response হলে browser বা frontend application JSON data ব্যবহার করতে পারে।

---

12. Complete Django Request Lifecycle Diagram

```
+----------------------+
|   Browser / Client   |
+----------------------+
           |
           | HTTP Request
           v
+----------------------+
|     Web Server       |
+----------------------+
           |
```


v
WSGI / ASGI
Request Middleware
URL Resolver
(urls.py)
View (views.py)
Django ORM
Database
Query Result
View Processes Data
Template Engine
(If HTML Is Needed)

```
+----------------------+
           |
           v
+----------------------+
|   HTTP Response      |
+----------------------+
           |
           v
+----------------------+
| Response Middleware  |
+----------------------+
           |
           v
+----------------------+
|     WSGI / ASGI      |
+----------------------+
           |
           v
+----------------------+
|     Web Server       |
+----------------------+
           |
           v
+----------------------+
|   Browser / Client   |
+----------------------+
```

এই diagram-এ database এবং template engine-কে সহজে বোঝানোর জন্য ধারাবাহিকভাবে দেখানো হয়েছে। বাস্তবে database query এবং template rendering view-এর প্রয়োজন অনুযায়ী হয়; সব request-এ দুটোই লাগবে না।

---

13. Real Example — Step by Step

ধরা যাক, user browser-এ request করেছে:

```
GET /products/5/
```

আমরা একটি product detail page তৈরি করতে চাই।

Step 1 — Browser

Browser request পাঠায়:

```
GET /products/5/
```

Step 2 — Web Server and WSGI / ASGI

Web server request গ্রহণ করে Django application-এর কাছে পৌঁছে দেয়।

Step 3 — Request Middleware

Django middleware request process করে।

উদাহরণ:

- Security checks
- Session processing
- Authentication processing
- CSRF checks, যেখানে প্রযোজ্য

Step 4 — URL Resolver

`urls.py`-এর pattern match হয়:

```
path("products/<int:id>/", product_detail)
```

Django `id=5` নির্ধারণ করে এবং `product_detail` view নির্বাচন করে।

Step 5 — View Execution

```
def product_detail(request, id):
    product = Product.objects.get(id=id)

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

Step 6 — ORM and Database

```
Product.objects.get(id=5)
```

Django ORM database query execute করে এবং product object ফেরত দেয়।

Step 7 — Template Rendering

```
render(
    request,
    "product.html",
    {"product": product},
)
```

Django template এবং product data ব্যবহার করে HTML তৈরি করে।

Step 8 — HTTP Response

View rendered response object return করে।

Step 9 — Response Middleware

Response middleware chain-এর মধ্য দিয়ে ফিরে আসে।

Step 10 — Browser

Browser final HTTP response গ্রহণ করে HTML page প্রদর্শন করে।

---

14. Request Flow vs Response Flow

Interview-এর জন্য এটি খুবই গুরুত্বপূর্ণ।

Request Flow

```
Browser
   ↓
Middleware 1
   ↓
Middleware 2
   ↓
Middleware 3
   ↓
URL Resolver
   ↓
View
```

Response Flow

```
View
   ↓
Middleware 3
   ↓
Middleware 2
   ↓
Middleware 1
   ↓
Browser
```

Shortcut:

- Request: Middleware order অনুযায়ী সামনে এগোয়।
- Response: Middleware response processing সাধারণত উল্টো ক্রমে চলে।

---

15. Why Does Middleware Order Matter?

Middleware order গুরুত্বপূর্ণ, কারণ একটি middleware অন্য middleware-এর তৈরি করা information-এর ওপর নির্ভর করতে পারে।

উদাহরণ:

```
MIDDLEWARE = [
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
]
```

এখানে `SessionMiddleware` আগে রাখা হয়েছে, কারণ `AuthenticationMiddleware` session information ব্যবহার করে user authentication পরিচালনা করে।

তাই middleware-এর order random নয়। ভুল order হলে application error বা unexpected behavior দেখা দিতে পারে।

---

16. Important Concepts — Quick Revision

Concept	**কাজ**
Browser	HTTP request পাঠায় এবং response গ্রহণ করে
Web Server	HTTP traffic গ্রহণ করে Django application-এর সঙ্গে যোগাযোগ করে
WSGI / ASGI	Server এবং Python applicatio

	n-এর মধ্যে interface
Middleware	Request ও response process করে
URL Resolver	URL pattern match করে সঠিক view নির্ধারণ করে
View	Application logic execute করে এবং response return করে
Model	Database data-এর structure নির্ধারণ করে
ORM	Python code দিয়ে database interaction সহজ করে
Database	Application data সংরক্ষণ করে

Template Engine	Template ও context data থেকে HTML তৈরি করে
HttpResponse	HTTP response তৈরি করে

---

17. Interview Questions and Answers

Q1. What is the Django Request Lifecycle?

Answer: The Django Request Lifecycle is the complete process in which an HTTP request travels from the browser through the web server and WSGI/ASGI interface into Django, gets processed by middleware, URL routing, and a view, and finally returns an HTTP response to the client.

Q2. What does the URL Resolver do?

Answer: The URL resolver matches the requested URL against URL patterns and determines which view should handle the request.

Q3. What is Django Middleware?

Answer: Middleware is a reusable processing layer that can process HTTP requests before the view executes and HTTP responses after the view returns.

Q4. Can Middleware Block a Request?

Answer: Yes. Middleware can return an HTTP response directly without allowing the request to reach the view.

Q5. What does Django ORM do?

Answer: Django ORM provides a Python-based interface for interacting with relational databases, allowing developers to perform database operations using models and QuerySets.

Q6. What does `render()` do?

Answer: The `render()` function combines a template with context data, renders the template, and returns an `HttpResponse` containing the rendered content.

Q7. What is the difference between WSGI and ASGI?

Answer: WSGI is the traditional synchronous interface for Python web applications. ASGI supports asynchronous application handling and protocols such as WebSockets.

Q8. Why is Middleware Order Important?

Answer: Middleware order matters because some middleware depends on information provided by other middleware, and the processing order affects request and response behavior.

Q9. Does Every Django Request Need a Database Query?

Answer: No. A view can return a response without accessing the database.

Q10. Does Every Django Request Need Template Rendering?

Answer: No. Templates are commonly used for HTML responses, but APIs can return JSON using `JsonResponse` or other response classes without rendering an HTML template.

---

18. Easy Memory Trick

মনে রাখার জন্য এই flow-টি practice করো:

```
Browser
   ↓
Web Server
   ↓
WSGI / ASGI
   ↓
Middleware
   ↓
URL Resolver
   ↓
View
   ↓
Model / ORM / Database
   ↓
Template (If Needed)
   ↓
HTTP Response
   ↓
Middleware
```

```
    ↓
Browser
```

Five Most Important Points

1. **Middleware:** Request ও response process করে।
2. **URL Resolver:** কোন view execute হবে তা নির্ধারণ করে।
3. **View:** মূল application logic execute করে।
4. **ORM:** Database interaction সহজ করে।
5. **Response:** Django থেকে server-এর মাধ্যমে browser-এ ফিরে যায়।

Interview-এর জন্য এক লাইনে:

The Django Request Lifecycle starts when a client sends an HTTP request and ends when the client receives an HTTP response after Django processes the request through the relevant middleware, URL resolver, view, and any required database or template operations.